

Day 7 Evidence Workbooklet

Ordered Validation and First-True Behavior

Engineering practice → ordered rules → first true branch → skipped branches → support route

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/42		Mastery check	secure / developing / needs support	

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the route should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Route a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an <code>if/elif/else</code> chain as an ordered list of checks where the first true branch wins.
Python focus	<code>if</code> , <code>elif</code> , <code>else</code> , first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, route assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if/elif/else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the route.
5. A reliable route note names the first true rule and any important skipped issue.

1 Read the ordered rules before tracing cases

[recognize](#)[predict](#)

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the route and the later rules are skipped.

```
1 kit_label = "S-22"
2 charge_percent = 76
3 calibration_label = "current"
4 borrower_initials = "MR"
5 damage_note = ""
6
7 if damage_note != "":
8     route = "repair bench"
9 elif charge_percent < 80:
10    route = "charge station"
11 elif calibration_label != "current":
12    route = "calibration bench"
13 elif borrower_initials == "":
14    route = "borrower log needed"
15 else:
16    route = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	
1	Which rule is checked second?	
1	Which rule is true for this kit?	
1	Which route is assigned?	

2 Trace first-true behavior

[trace](#) [explain](#)

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and route
2	no	76	current	MR	
2	cracked case	76	expired	MR	
2	no	91	expired	MR	
2	no	91	current	blank	

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3 Explain skipped branches

[explain](#)[debug](#)

A kit may have more than one problem. Ordered routing decides which problem is handled first.

```
1 charge_percent = 72
2 calibration_label = "expired"
3 damage_note = ""
4
5 if damage_note != "":
6     route = "repair bench"
7 elif charge_percent < 80:
8     route = "charge station"
9 elif calibration_label != "current":
10    route = "calibration bench"
11 else:
12    route = "release kit"
```

Pts.	Prompt	My response
2	Which route is assigned?	
2	Is the calibration rule also true for this kit? Explain.	
2	Why does the calibration branch not run?	
2	What would be dangerous about reporting only the final route without the skipped issue?	

4 Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif damage_note != "":
    route = "repair bench"
else:
    route = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	
2	Give a kit case that would be routed unsafely by the proposed order.	
2	What route should that case receive?	
2	Write one sentence explaining the rule-order risk.	

5 Constrained edit of one rule

[implement](#)[debug](#)

Only edit the calibration rule below. Do not rewrite the whole route.

```
elif _____:
    route = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	
2	Explain why the rule should compare to "current".	
2	Give one calibration label that should route to the calibration bench.	
1	Name one later branch that would be skipped if this rule is true.	

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single <code>if/else</code> . Use first true and skipped branch in your explanation.
2	Circle the support route you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: _____

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a route when the selected action does not match the rule intention.

What stays fixed

Keep the `def` line, parameter names, and exact returned strings. You are not designing a function definition. You are completing the indented decision body that the notebook already provides. This page adds no points; the workbook score remains unchanged.

```
def badge_sheet_plan(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if _____:
        return "one sheet"
    elif _____:
        return "two sheets"
    else:
        return "print batch"
```

Total badges	Expected route	First true condition	Why this is a boundary case
8	one sheet		
9	two sheets		
16	two sheets		
17	print batch		

Exact-output contract

The visible checks compare exact returned strings. Treat "one sheet", "two sheets", and "print batch" as fixed output labels, not wording to improve.